

MI ágenssek Unity környezetben

Csapatban működő ágenssek fejlesztése Unity ML-Agents használatával (Diplomaterv 1)

Készítette: Nagy Dániel

Konzulens: Dr. Vaitkus Márton

A projekt GitHub repositoryja: <https://github.com/NDani23/AI-Warfare>

Bevezetés

A mesterséges intelligencia kutatása napjainkban (mint ahogy már párszor a történelem során) ismét a figyelem középpontjában van az informatika világában. A neurális hálókat használó gépi tanulás, vagy mélytanulás egyre több területén bukkan fel az iparnak és a tudománynak.

A mélytanulás egyik szélesebb körben elterjedt formája a megerősítéssel tanulás, amely többek között a különböző autonóm rendszerek fejlesztésében tűnik ígéretes megoldásnak. Például az egyre nagyobb hírnévnek örvendő önvezető autók működésében is központi szerepet tölt be ez a technológia.

Egy önállóan működő, a feladatát jól elvégezni képes megerősítéssel tanuló rendszer kifejlesztése már önmagában is egy nagy kihívás. A problémát azonban tovább bonyolítja, ha az ágenseknek csapatban dolgozva, egymással együttműködve kell megoldania egy adott feladatot. Az ágensek közötti csapatmunka olyan extra kihívásokat állít a fejlesztők elé, aminek megoldására a hagyományos megerősítéssel tanulási algoritmusok és módszerek sokszor nem nyújtanak egyértelmű megoldást. Természetesen az ilyen jellegű feladatok megoldására használható módszerek területén is folyamatosan zajlanak a kutatások, és gyors ütemben jelennek meg újabb és újabb megoldások. A Unity által fejlesztett ML-Agents nevű megerősítéssel tanulási keretrendszer is implementálja ezen megoldások egy halmazát.

A csapatban együttműködő ágensek mellett elképzelhető egy másik, szintén nagy kihívást jelentő szcenárió is, amikor az a cél, hogy az önállóan, vagy csapatokban működő modellek képesek legyenek egymás ellen dolgozni. Bár a való életben használt/a külvilággal érintkező autonóm rendszerek esetében ez a felhasználási eset valószínűleg nem annyira releváns, azonban egy szimulált környezetben, például egy videójátékban gyakran szeretnénk elérni ilyen jellegű viselkedést. Ennek megvalósítása is jelentősen növeli a tanítás komplexitását és felvet újabb megoldásra váró problémákat.

Az önálló laborom során részletesen megismerkedem a fentebb említett feladatokkal és az ezek megoldására létező megoldási módszerekkel a Unity által fejlesztett ML-Agents keretrendszert használva.

A projekt célja

A Diplomaterv 1 tantárgy keretein belül a célom ezzel a projekttel, hogy a Unity ML-agents-en keresztül megismerkedjek a csapatokban és egymás ellen dolgozó ágensek fejlesztésének speciális módszereivel és megoldásaival egy szimulált játék környezetet használva.

A szimuláció/játék lényege, hogy ágensek 2 csapatra osztva harcolnak egymás ellen különböző harci járműveket irányítva. A projekt jelenlegi formájában még csak tankokat irányítani képes ágenseket tartalmaz. Az ágensek előre felépített pályákon harcolnak és céljuk, hogy a csapattársaikkal együtt dolgozva, tankokat és egyéb harci járműveket irányítva megsemmisítsék az ellenfél csapat ágenseit és képesek legyenek teljesíteni különböző játékmódok kihívásait. A projekt jelenleg 1 pályát tartalmaz, a tankokat irányító ágensek pedig még csak a hagyományos Conquest¹ játékmódot ismerik. A csapatok pontokat kapnak az ellenfél ágenseinek elpusztításáért és a játékmód specifikus feladatok elvégzéséért. Az a csapat nyer, amelyik hamarabb elér egy megadott ponthatárt. Egy csapatot jelenleg 5 ágens tesz ki.

Az ágenseknek tehát több, viszonylag nehéz összetett viselkedést is meg kell tanulniuk. Az első lépés természetesen magának a jármű irányításának elsajátítása, ami önmagában is elég bonyolult, sokrétű feladat. Ezen felül meg kell tanulniuk a pályán való közlekedést, navigációt (például akadályok kikerülése) valamint a fő feladat elvégzését is, tehát az ellenfelek kiiktatását és a kontroll pont elfoglalását, mindezt csapatban együtt dolgozva.

¹ A csapatok célja a pálya közepén lévő kontroll pont elfoglalása és megtartása.

A Tank ágens működése

Ebben a részben térek ki magának a tanknak az irányítására, az ágensek megfigyeléseire és a lehetséges akciókra.

A tank irányítása

Az ágensek által irányított tank alapvetően arcade jellegű, mégis kicsit lomha, nagy tömegű jármű, ami a szimulációban is érződik. A lehetséges inputokat tekintve nagyon hasonló egy tradicionális, videójátékokból ismert tank működéséhez. A jármű képes előre/hátra haladni, kanyarodni, valamint a tankoknál megszokott módon akár helyben is fordulni. Emellett a tank törzsének orientációjától függetlenül mozgatható maga az ágyú függőleges és vízszintes irányba egyaránt. Az ágyú függőleges mozgatása (fel-le emelése) természetesen le van korlátozva a tank törzséhez viszonyítva. Ezek mellett az ágensnek persze tüzelni is van lehetősége. Egy lövést követően 3 másodperc múlva lehet újra tüzelni.



1. ábra: A tank

A fent leírt viselkedést tükrözve az ágenseknek minden körben 5 különböző akciót kell megtenniük, ezek rendre:

folytonos akció:

- lövegtorony vízszintes irányú mozgatása (célzás)

diszkrét akció:

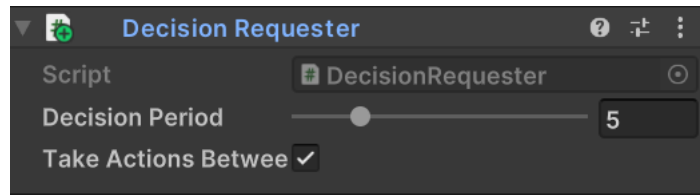
- gázadás: [hátra, null input, előre]
- fordulás/kanyarodás: [balra, null input, jobbra]
- lövés: [null input, lövés]
- ágyú függőleges irányú mozgatása: [le, null input, fel]

Látható, hogy vegyesen használok diszkrét és folytonos akciókat is. Eredetileg kizárólag diszkrét akciókat szerettem volna használni, hiszen ezekre jóval gyorsabban, könnyebben képesek tanulni az ágensok. A lövegtorony mozgatásánál azonban fontos, hogy az ágensok finom mozgásokat tudjanak végezni, és pontosan célba tudják venni az ellenfél tankjait. A 2. ábrán láthatóak a beállított akció paraméterek.

Actions	
Continuous Actions	1
Discrete Branches	4
Branch 0 Size	3
Branch 1 Size	3
Branch 2 Size	2
Branch 3 Size	3

2. ábra: akció paraméterek

Az akciók kapcsán meg kell még jegyezni, hogy az ágensek DecisionRequester komponensének DecisionPeriod paraméterét ebben az esetben az alapbeállításon, azaz ötön hagytam. Ez egy olyan paraméter, amit



3. ábra: DecisionRequester komponens

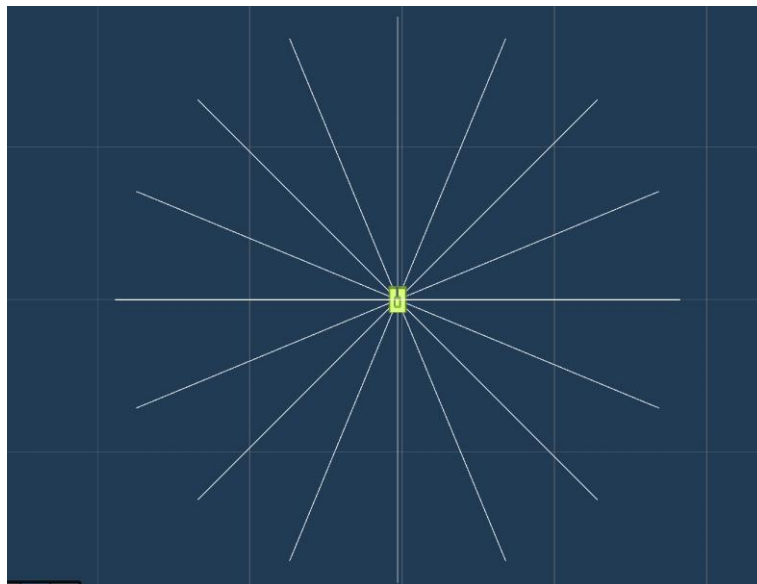
az előző fél éves projektben nem kezeltem teljesen jól, hiszen annál az alkalmazásnál 1-re állítottam az értékét gondolva, hogy ezáltal javul az ágensek reakció ideje. Bár ez az elgondolásom nem feltétlenül volt helytelen, azonban nagyban megnehezítette az ágensek tanulását. A 3. ábrán látható „Decision Period” beállítás például azt jelenti (a „Take Actions Between Decision” igazra állítása mellett), hogy az ágens minden 5. életciklus körben fog csak újabb akciókat választani, és a következő 5 körben ezeket az akciókat fogja ismételni. A tanítás szempontjából ez azt jelenti, hogy egy-egy akció választásnak sokkal nagyobb hatása lesz az ágens állapotaira, ami egyértelműsíti az adott akció hatásait és megkönnyíti a tanulást. Ráadásul ez az érték az ágens reakcióidejét sem korlátozza nagy mértékben (Mivel a projekten belül 0.02 másodpercenként történik a fizikai szimuláció frissítése így az ágensek reakcióideje még így is $0.02 * 5 = 0.1$ másodperc, ami a tankok viszonylag lassú mozgásával együtt bőven elegendő a jó teljesítményhez.)

Megfigyelések

Az ágensek térbeli tájékozódását elsősorban a Unity beépített RayPerceptionSensor3D komponense segíti. A szenzor rendszer felépítése:

A tank törzsén:

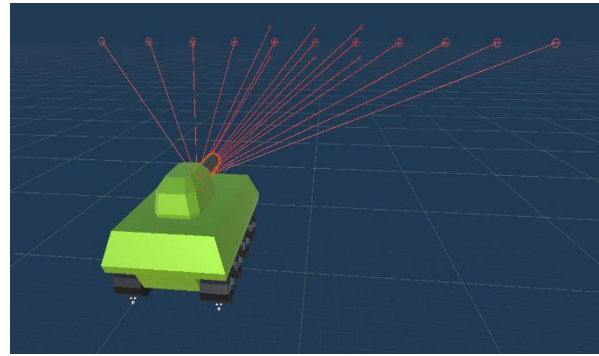
- 360 fokos szögben a jármű körül viszonylag rövidebb sugarak a közeli akadályok, objektumok alacsony felbontású érzékelésére.



4. ábra: Érzékelők a tank törzsén

Az ágyún:

- 1 db sugár közvetlenül az ágyúból indulva, ez alapján céloz az ágens
- Előre vízszintes 32 fokos szögben 10 darab sugár. Ez alapján tudja beazonosítani az ágens a „látómezőjébe eső” akadályokat, csapattársakat, ellenségeket.
- 3-3 sugár az előző szenzor komponenshez képest felfelé és lefelé irányítva. Feladata hasonló az előző komponenshez, csak függőleges irányban.



5. ábra: Érzékelők az ágyún

A sugarak képesek felismerni a „YellowAgent”, „RedAgent” tageket. A projekt korábbi verziójában a felismerhető tagek között volt még a „ControllPoint” is, azonban a vektor alapú megfigyelések mellett ez az információ redundáns, elhanyagolható és elhagyása nagy mértékben csökkenti a háló inputjainak számát. A legtöbb sugarakból érkező megfigyelést hármassával vagy ötösével („stackelve”) adom át az ágensnek. Ezt azt jelenti, hogy az ágens nem csak az aktuális frame-hez tartozó információkat kapja meg, hanem az elmúlt 2-4 frame-hez tartozókat is. Ha mindig csak a RayPerceptionSensor-ok aktuális értékeit kapná meg az ágens, akkor semmi információ nem állna rendelkezésére ahhoz, hogy a többi objektum mozgási dinamikáját érzékelje (pl.: merre/milyen gyorsan halad az ellenség).

A sugarakon kívül az ágensek kapnak pár tradicionális vektor alapú megfigyelést is:

- A tank sebessége (negatív előjellel amennyiben tolat az ágens)
- A lövés cooldown² ideje
- Az ágens életerej
- Az ágyú jelenlegi orientációja (előre mutató vektorja)
- Az aktuális cél felé mutató egységvektor (Ez játékmódonként eltérő lehet, a jelenlegi conquest játékmódban ez a KontrollPont)
- Az aktuális cél és az ágens távolsága
- A játékosról érkező parancs numerikus azonosítója (még nincs kihasználva)
- A meccsből hátralévő idő.
- Az ágens saját csapatának pontszáma
- Az ellenséges csapat pontszáma
- A kontroll pont aktuális státusza (melyik csapat tartja irányítása alatt)

Az ilyen vektor alapú megfigyelések esetében van pár bevett gyakorlat/javaslat, amit érdemes betartani annak érdekében, hogy megkönnyítsük az ágensek tanulását. Az egyik ilyen javaslat, hogy minden megfigyelést normalizáljunk le, tehát az értékük 0 - 1 vagy -1 – 1 közé essen. Ugyanis, ha nagyságrendekkel eltérnek az egyes megfigyelések egymástól, az megzavarhatja, esetleg prioritizálásra készíti az ágenszt. A normalizálást akár a config file-ból is be lehet állítani, azonban véleményem szerint egyértelműbb, ha ezt magunknak tesszük meg a forráskódban. A

² Két lövés között legalább 3 másodpercrek kell eltelnie.

normalizáláson kívül azzal is segítem az ágensek tanulását, hogy minden esetben, amikor valamilyen irányt (pl.: a kontroll pont felé mutató vektor) adok át megfigyelésként, akkor az adott vektort áttranszformálok az ágens saját koordináta rendszerébe. Így az ágenseknek nem kell megtanulnia a globális irányokat, hiszen ez a működésük szempontjából nem releváns. A globális irányok helyett csak a saját orientációjukhoz képesti irányt kapják meg. A Unity-n belül erre a transzformációra egyszerű megoldást nyújt az „InverseTransformVector” metódus.

```
memory:
  use_recurrent: true
  sequence_length: 64
  memory_size: 128
```

6. ábra: Memória beállítás a config file-ban

A fentebb említett sugár és vektor alapú megfigyelések segítségével már viszonylag jó teljesítményt tudna elérni az ágens, azonban van még egy dolog, ami jelentős hátrányt jelentene a pusztán ezeket a megfigyeléseket használó ágenseknek például egy humán játékos ellen.

Ez nem más, mint a memória. Gondoljunk például arra a szituációra, amikor egy célba vett ellenséges tank fedezék mögé húzódik, és ezzel kikerül a játékos látómezőjéből. Egy humán játékoskal ellentétben egy memória nélküli RL modell ebben a pillanatban teljesen el fogja felejteni, hogy ott volt egy ellenfél, így persze nem fogja követni azt, nem próbál meg mögé vágni stb... Bár a Unity ML-Agents rendelkezik memória támogatással (6. ábra), az ezzel folytatott kísérleteim során az ágensek tanulása nem volt eredményes, ráadásul a tanulási idő is sokszorosára nőtt. Valószínűleg további próbálkozásokkal és a beállítás részletesebb megismerésével orvosolható lett volna a probléma,

```
RayPerceptionInput spec = aimSensor.GetRayPerceptionInput();
RayPerceptionOutput obs = RayPerceptionSensor.Perceive(spec);
if (obs.RayOutputs[0].HitTagIndex == 0)
{
    envController.EnemyDetected(obs.RayOutputs[0].HitGameObject.transform.parent.gameObject, this.team);
}
```

7. ábra: Kódrészlet RayPerceptionSensor kimenetének kinyeréséhez

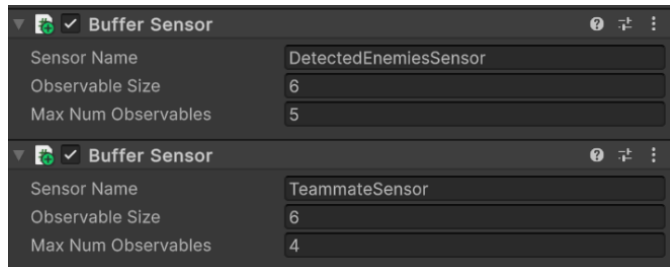
én azonban egy másik megközelítést választottam. A kompetitív videójátékokban egy elterjedt mechanizmus, hogy lehetőség van ellenséges játékosokat, karaktereket megjelölni („spotolni”) a csapattársak számára. Én ezt a megközelítést szerettem volna modellezni az ágensek számára ezzel megkerülve a fentebb leírt problémát.

Mindkét csapat rendelkezik egy-egy adatszerkezettel, ami tárolja az éppen megjelölt ellenségek halmazát. A megfigyelések átadásánál az ágensek kapnak minden megjelölt ellenséghez kapcsolódóan információkat, ezek a következők:

- Egységvektor az ellenséges ágens irányába
- Távolság az ellenséges ágenstől
- Az ellenséges ágens típusa (pl.: tank, helikopter)
- Az ellenséges ágens életeréje

Egy ellenséges tank akkor kerül be a megjelöltek adatszerkezetébe, amikor egy csapattag célba veszi azt. Ez lényegében azt jelenti, hogy a célzásra használt RayPerceptionSensor találatot jelez egy ellenséges csapat tagével ellátott objektummal. Ennek a konkrét megvalósítása kissé körülményes, ezt a 7. ábrán látható kódrészlet szemlélteti.

Egy detektált ellenséges ágens 15 másodperc elteltével kitörlődik a felderítettek adatszerkezetéből. Ez a megoldás felvet egy kis nehézséget, mégpedig, hogy az éppen felderített ellenségek száma folyamatosan változik. A hagyományos vektor alapú megfigyeléseknél azonban fix számú adatot tudunk átadni a modellnek. Szerencsére a Unity ML-Agents által implementált BufferSensor komponens pontosan ezt a feladatot képes megoldani. Ennek használatát szemlélteti a 7. és 8. ábra.



8. ábra: BufferSensor komponensek

```
foreach (var agent in detectedEnemies.Keys.ToList())
{
    Vector3 dir = transform.InverseTransformVector(Vector3.Normalize(agent.transform.localPosition - transform.localPosition));
    float[] dirObs = { dir.x, dir.y, dir.z };
    detectedEnemiesSensor.AppendObservation(dirObs);
}
```

9. ábra: Kódrészlet BufferSensor használatára

A projekt előző verziójában a tank ágensok viselkedése nem mutatott túl sok csapatmunkát, Ennek megoldása volt az egyik fő célom erre a félévre. A csapatmunkára való ösztönzés érdekében a projekt új verziójában az ágensok már a csapattársaikról is folyamatosan kapnak információkat (pozíció, típus, életerő). Ezeket az információkat is egy BufferSensor komponensen keresztül adom át az ágenseknek, hiszen az aktuálisan életben lévő csapattársak száma is változik.

Érdekes eredmény, hogy ez az egyszerű kibővítése a megfigyeléseknek elég volt ahhoz, hogy az ágensok sokkal inkább együtt mozogjanak és már egyszerűbb csapatszintű stratégiák is kezdtek megjelenni:

Például a tanítás közben észrevettem, hogy abban az esetben amikor az ellenséges csapat minden csapattársát kilőtte egy ágens, akkor a szóban forgó tank ágens nem rontott be a kontroll pontra egyedül, hanem sok esetben megvárta amíg a csapattársai újra élednek.

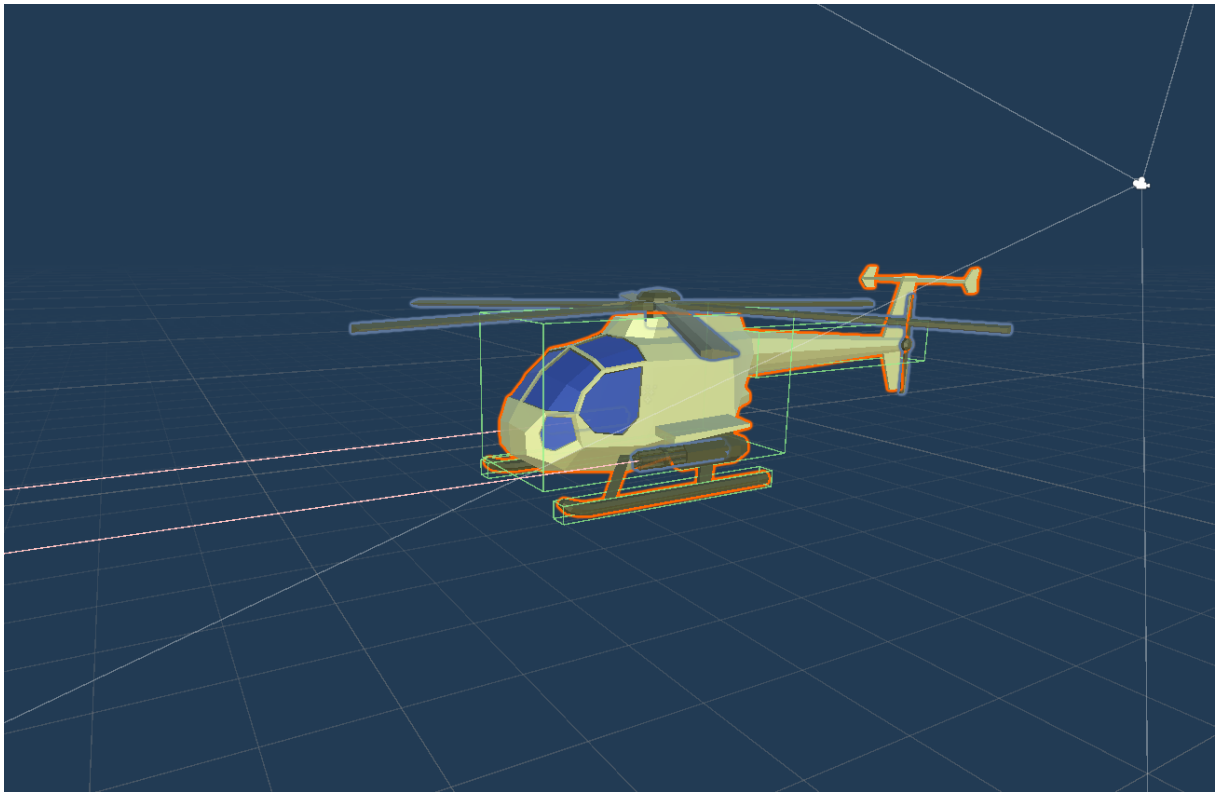
Ez azért is érdekes, mert ezzel az új megfigyeléssel csak a csapatmunka lehetőségét tettem elérhetőbbé az ágenseknek, jutalmakkal nem tereltem őket a csapatmunka felé.

Helikopter ágens

A projekthez hozzáadásra került egy újfajta jármű, a harci helikopter. Ez egy gyors mozgású, de kisebb tűzerővel rendelkező jármű, aminek két fő feladata lesz a későbbiekben:

- Légi támogatás nyújtása a tankoknak
- Ellenséges egységek felderítése

Az ágens 2 darab gépágyúval rendelkezik a törzs két oldalán, amelyek folyamatosan képesek tüzelni, azonban képesek a túlmelegedésre. Ez azt jelenti, hogy ha bizonyos ideig folyamatosan tüzel az ágens, akkor a fegyverek túlmelegsznek és adott időnek el kell telnie amíg újra tüzelni tudnak. jelenleg ehhez a járműhöz még nincs betanított ágens, csak az előkészületek és a projektbe történő integrációra került sor.



10. ábra: Helikopter prefab

Ez az ágens 2 folytonos akcióval:

- pitch
- yaw

És 3 diszkrét akcióval fog rendelkezni:

- roll
- throttle (felhajtóerő mértéke, vagy propellerek sebessége)
- tüzelés

Ennek az ágensnek a betanításánál külön kihívás lesz, hogy itt már egyszerűen az életben maradás sem triviális feladat, hiszen amikor az ágens ütközik valamivel, akkor életerőt vonok el tőle a becsapódás sebességéhez mérten.

Megfigyelések:

A hagyományos vektor megfigyelések többsége ugyanaz, vagy nagyon hasonló a tanknál látottaknál, kiegészítve azokat persze a helikopter irányításához szükséges dolgokkal:

- roll, pitch, yaw értékek
- magasság
- fegyver túlhevülési mutatója

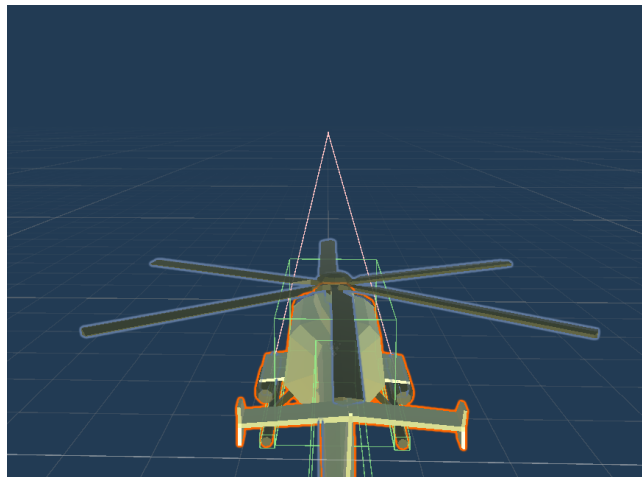
A tankok, bár 3D-s objektumok igazából csak a vízszintes síkban játszá a játékot (a pályák teljesen laposak) így a térbeli tájékozódáshoz elegendőek a RayPerceptionSensor-ok által nyújtott megfigyelések. A helikopter ágensnél már más a helyzet, hiszen ez a jármű már ténylegesen a 3 dimenziós tér minden irányában képes mozogni és képesnek kell lennie

navigálni. Ráadásul előre felé viszonylag távolra kell, hogy „lásson”. Ezek miatt az okok miatt a RayPerceptionSenszorok már sajnos nem elegendőek, vagy csak átláthatatlanul nagy számban tudnának kellő részletességű információt szolgáltatni az ágens környezetéről.

A leg kézenfekvőbb megoldás erre a problémára a kamera alapú (vizuális) megfigyelések használata. A helikopter ágens prefabje tartalmaz egy, a helikopter pozíciójához képest fix pozíciójú és orientációjú kamerát. A kamera az ágens mögött előre felé nézve helyezkedik el, így lényegében egy hagyományos 3rd person view nézetet megvalósítva.

A vizuális megfigyelések azonban felvetnek pár problémát. Az első és legnyilvánvalóbb probléma a teljesítmény, egy ilyen megfigyelés nagyon nagy mértékben megnöveli a háló bemeneteinek a számát, ami különösen problémás lehet egy ilyen, több ágenszt tartalmazó játékkörnyezetben. Ezt a problémát többek között a felhasznált képi bemenet felbontásának csökkentésével enyhítem. A neurális hálóba táplált kép 64x64-es, ami bár nyilván pixeles, de arra bőven elégnek kell lennie, hogy az ágens navigálni tudjon a pályán és felismerje a legfontosabb objektumokat. Ez a felbontás nem elegendő azonban ahhoz, hogy pontosan tudjon célózni a fegyverekkel az ágens, így a célzáshoz biztosítok még fegyverenként 1-1 darab RayPerceptionSensort-is, a 11. ábrán látható módon.

A másik probléma az a lentebb ismertetett szimmetrikus megfigyelések problémája. A RayPerceptionSenszorok esetében egyszerűen a tagek felcserélésével megoldható az a probléma, hogy mindkét csapat valóban az ellenséges ágenseket lássa ellenségként. Egy vizuális képnél már nem ennyire egyértelmű a megoldás, és ennél az ágensnél sem szeretnék az expliciten megadott teamID megadására hagyatkozni. A probléma tehát az, hogy amíg az egyik csapat ágenseinek azt kéne megtanulnia, hogy a piros objektumokat támadja, addig a másik csapatnál pont ellenkezőleg, a sárga objektumokat kell támadni, mindezt úgy, hogy az ágensnek fogalma sincs arról, hogy melyik csapatba tartozik.

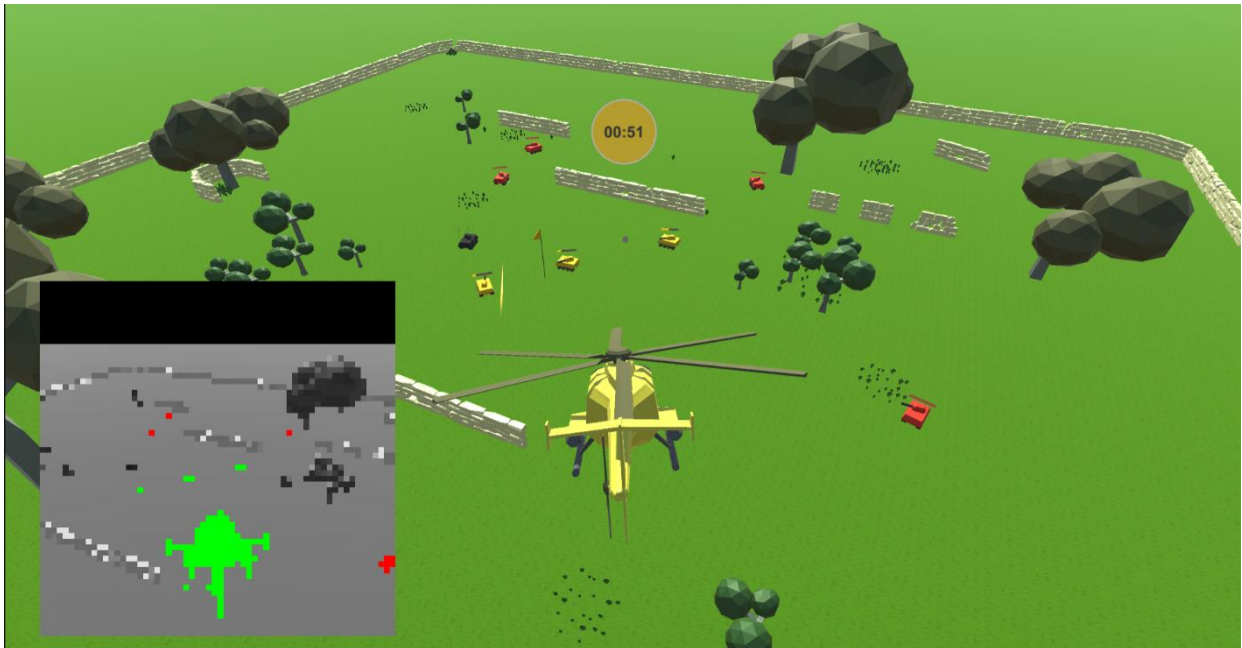


11. ábra: Célzáshoz használt sugarak

Megpróbáltam tehát a RayPerceptionSensor-oknál látott szimmetrikus megfigyelés gyakorlatát átültetni vizuális megfigyelésekre. Azt kell elérni, hogy mind a két csapat helikoptere ugyanolyannak lássa (ugyanolyan színűnek) az ellenségeket és csapattársakat, például az ellenséges ágensek minden piros, a csapattársak pedig mindig zöld színben jelenjenek meg. Ezt úgy érem el, hogy az ágenshez tartozó kamera renderelési folyamata előtt lecserélem a színtérben szereplő ágensek material-ját, az „onPreRender” beépített metódus segítségével, a renderelés végeztével pedig visszaállítom azokat az eredeti értékükre. Az így kapott képet még egy saját shader segítségével utólag fel is dolgozom olyan módon, hogy csak a színtérben szereplő ágensek jelenjenek meg színesen, a környezeti elemek színeit pedig szürkeárnyalatossá konvertálom. Ennek több előnye is van:

- Az ágensnek talán segítséget nyújt abban, hogy kiszűrje a képből a lényegét, azaz a színtérben szereplő többi ágens.
- A különböző stílusú pályák eltérő színvilága kevésbé zavarja össze az ágens.
- Mivel a képen így csak lényegében piros és zöld színeket használlok, az elkészült képet elég egy 2 csatornás képként beadni a neurális hálónak, ezzel nagy mértékben lecsökkentve az inputok számát.

A 12. ábrán az látható, hogy a hagyományos gameplay-hez képest az ágensek hogyan fogják látni a játékot.



12. ábra: Vizuális megfigyelés

Kooperatív viselkedés megvalósítása

Ebben a részben azt fogom részletesebben kifejteni, hogy a Unity ML-Agents milyen támogatásokkal rendelkezik a kooperatív ágensek tanításához, és mikre kell odafigyelni a kívánt működés eléréséhez.

SimpleMultiAgentGroup

A Unity ML-Agents rendelkezik egy beépített osztállyal az egy csapatba tartozó ágensek közös kezeléséhez, ez a SimpleMultiAgentGroup osztály. Az egy csapatba tartozó ágenseket a szimuláció kezdetén hozzá kell rendelni a csapathoz tartozó MultiAgentGrouphoz a „RegisterAgent” metódus segítségével. Az osztály lehetővé teszi a csapatszintű jutalmak kiosztását, illetve az epizódok csapat szintű kezelését. Mivel ez az objektum nem köthető egy bizonyos ágenshez sem, érdemes egy külön, az egész környezetet vezérlő osztályba szervezni (Ebben a projektben ez az EnvController).

Csapatban dolgozó ágensek esetében általában csapatszintű jutalmakat szokás definiálni. A MultiAgentGroup „AddGroupReward” metódusával van lehetőségünk csapatszintű jutalmak kiosztására. Az ilyen módon definiált jutalmakat a csapat minden tagja megkapja a saját tevékenységétől függetlenül. Amikor MultiAgentGroup-okkal dolgozunk, akkor az egyéni CumulativeReward metrika helyét a GroupCumulativeReward veszi át a tanítás során (Fontos,

hogyan ez csak a későbbiekben tárgyalt POCA trénerrel használva működik). Ez teljesen logikus, hiszen az ilyen esetekben azt szeretnénk elérni, hogy minden ágens a csapat érdekében cselekedjen, és ilyenkor az egyéni jutalmak szerepe másodlagos. Összefoglalva tehát a POCA trénerrel és MultiAgentGroup-ot használva a tanítás során a cél a GroupCumulativeReward maximalizálása. Fontos megjegyezni azonban, hogy ilyen esetben is van lehetőségünk egyéni jutalmakat kiosztani az ágenseknek, így rávezetve azokat az elvárt viselkedésre, sőt a POCA trénerrel használva elméletben (dokumentáció szerint) az ágensek törekednek arra is, hogy maximalizálják egymás egyéni jutalmait. Általános javaslat viszont, hogy kooperatív ágensek tanításánál a csapatszintű jutalmakon legyen a hangsúly.

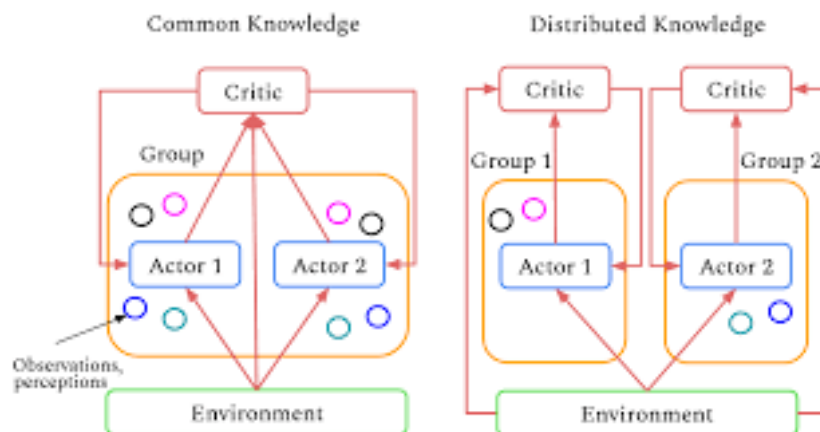
A MultiAgentGroup-on keresztül tudjuk vezérelni a csapat közös epizódjait is. Ez lényegében azt jelenti, hogy az epizódokat mindig a MultiAgentGroup osztály „EndGroupEpisode” vagy „GroupEpisodeInterrupted” metódusaival zárjuk le. Itt ragadnám meg az alkalmat, hogy kifejtsem a két metódus közti különbséget. Látszólag mind az „EndGroupEpisode” és a „GroupEpisodeInterrupted” is véget vet az adott epizódnak/játék körnek és meghívásuk után új kör fog kezdődni, a háttérben azonban kissé más hatást fejtenek ki a tanulási folyamatra. Az EndGroupEpisode valóban lezárja az aktuális epizódot és a tanított ágensek felé azt fogja jelezni, hogy az aktuális játékkör vége a saját tevékenységeiknek következménye. Ezzel ellentétben a GroupEpisodeInterrupted meghívásánál az ilyen módon lezárt epizódot a tréner nem fogja teljes értékű epizódnak tekinteni (például a későbbiekben tárgyalt ELO számítás nem is aktiválódik ilyen esetben). Ez a metódus hasonló hatást fejt ki, mint amikor az epizód lépésszáma eléri az ágenseknél definiált „MaxStep” értéket. A két metódus felhasználását tekintve abban az esetben kell az EndGroupEpisode-ot meghívni, amikor az aktuális körnek azért lett vége, mert az ágensek elérték valamilyen célt, vagy éppen elbukták a feladatot, tehát a kör vége közvetlenül az ágensek akcióinak a következménye. A GroupEpisodeInterrupted metódus tipikus felhasználási esete, amikor az adott játék kör elért valamilyen általunk definiált lépésszámot, vagy időkorlátot. Fontos, hogy MultiAgentGroup-ban lévő ágensek esetében a Max Step paraméter értékét nullára kell állítani.

A MA-POCA tréner

A hagyományos SAC vagy PPO trénerrel az egyéni CumulativeReward metrika maximalizálására tanítják be az ágenseket. Emiatt ezek a trénerrel kooperatív, csapatban dolgozó ágensek tanítására nem optimálisak. A Unity ML-Agents implementál egy harmadik, kifejezetten csapatban dolgozó ágensek tanítására szolgáló trénerrel is, ez a Multi-Agent Posthumous Credit Assignment (röviden MA-POCA) tréner.

A POCA tréner a korábban említett GroupCumulativeReward alapján tanítja a csapatokat és számos hasznos funkciót valósít meg, amelyek nagyban segítenek kooperatív ágensek fejlesztésénél. Ennek a trénernek a sajátossága, hogy az ágensek tanítása mellett karbantart/tanít egy másik, rekurrens neurális hálót is (centralized critic) ami egyfajta edzői szerepet tölt be egy adott csapat ágensei fölött.

A GroupReward-ok használata alaphoz felvetne pár kérdést és problémát. Képzeljük el azt a szituációt, amikor egy adott ágens egy csapat epizód során semmit nem tett a csapat sikeressége érdekében (sőt akár még hátráltatta is a csapatát) viszont a csapat többi tagjának jó teljesítménye elegendő volt ahhoz, hogy a csapat sikeresen teljesítse a feladatot. Ebben az esetben, mivel a GroupReward minden csapattagnál megegyezik, a nem kívánt viselkedést mutató ágens is jutalmazni fogjuk, ami nyilván nem szerencsés a tanítási folyamatra nézve. Ezt a problémát hivatott megoldani a csapat központi critic modellje. A központi „edző” rálát minden csapattag állapotára, megfigyeléseire és feladata, hogy ezek alapján megállapítsa, hogy egy adott ágens tevékenysége mekkora mértékben járult hozzá az egész csapat sikerességéhez és ennek függvényében jutalmazza őket.



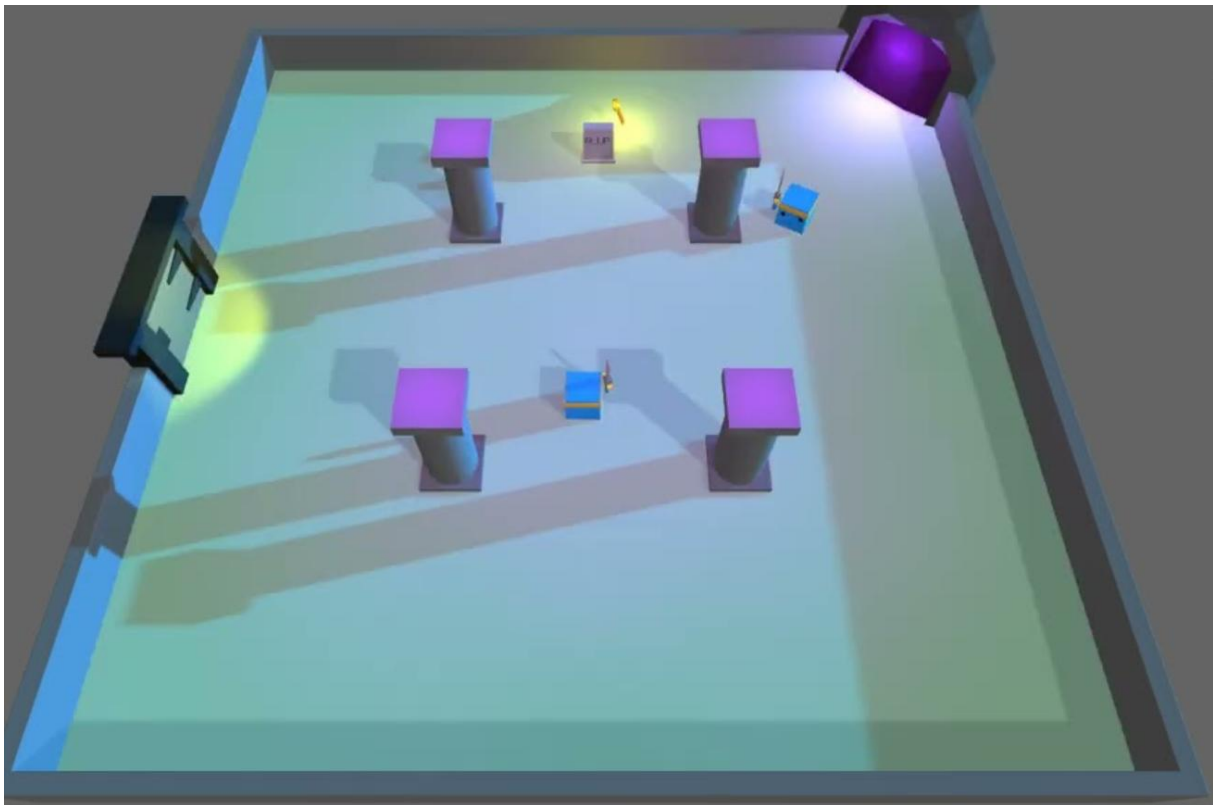
13. ábra: POCA tréner vázlatos működése

Mivel a központi critic az összes csapattag megfigyeléseit figyelembe veszi, így a tanítás során a különböző ágensek megtanulnak jó döntéseket hozni a csapattársaik helyzetének függvényében.

Halott ágensek kezelése

A kooperatív viselkedésből és a játék jellegéből adódóan felmerül a kérdés, hogy mi a legjobb módszer a halott ágensek kezelésére. A játékban, ha egy ágens élete 0-ra csökken, meghal és 10 másodperc múlva újra él a saját csapatának induló területén. Ez bonyodalmakat okoz, hiszen ebben az esetben például nem lehet véget vetni az aktuális epizódnak, hiszen a többi ágens még életben van, így valahogy meg kell oldani azt, hogy az ágensek jól kezeljék ezt a helyzetet.

A Unity dokumentációja alapján a MA-POCA tréner támogatja a csapattagok epizód közbeni deaktiválását olyan módon, hogy elméletileg a deaktivált ágensek is megkapják a csapatszintű jutalmakat és így akár önfeláldozó viselkedést is képesek megtanulni a csapat érdekeit szem előtt tartva. Ezt mutatja a Unity csapata által készített DungeonEscape példa projekt is, amit a 11. ábra szemléltet. Ezzel a megállapítással kissé szembe megy az a tapasztalatom, hogy deaktiválást követően az adott ágens már nem lesz része a csapat MutliAgentGroup-jának, amin keresztül a csapatszintű jutalmakat meg tudná kapni. Fontos még megjegyezni, hogy a saját projektemmel ellentétben a DungeonEscape nem használ Self-Play-t. A self-play használata további bonyodalmakat vet fel, hiszen ebben az esetben az ágensek teljesítményének nyomon követésére a Unity egy saját ELO rendszert használ, ami az adott ágens utolsó kapott jutalmának előjeléből következtet arra, hogy az ágens győztesként vagy vesztesként került ki az adott epizódból, az ágensek deaktiválása pedig véget vet annak saját epizódjának. Igazából a unity dokumentációja nem tér ki pontosan rá, hogy ilyen esetekben az ELO számítás hogyan működik.



14. ábra: DungeonEscape példa projekt

A fentebb említett kétségek és problémák miatt más megközelítést választottam a halott ágensek kezelésére.

Amikor egy ágens lelőnek, a következő 10 másodpercben ignorálok minden akciót, amit az adott ágens tesz. Emellett, az ágens megfigyeléseit kinullázom és a csapattársak sem fognak megfigyelést kapni az adott ágens helyzetéről. Fontos figyelni rá, hogy ebben az esetben az ágens azonosító tageket is le kell cserélni (szimplán untagged-re állítom), ellenkező esetben a többi ágens RayPerceptionSensor megfigyelései ugyanolyan élő ágensként érzékelnék a halottakat. Miután letelik a 10 másodperc, a halott ágens tageit visszaállítom, és visszahelyezem az ágens a kiinduló pozíciójába.

Ez a megközelítés talán okozhat egy kis bizonytalanságot az ágensek viselkedésében, viszont a tapasztalataim azt mutatják, hogy ez nem befolyásolja az ágensek működését, „újja éledés” után is jó viselkedést mutatnak. Ezzel a megoldással a halott ágensek biztosan megkapják a csapatszintű jutalmakat, illetve az ELO számítás is biztosan jól fog működni, hiszen az ágensek epizódjának nem vetek véget meccs közben.

Ellenséges ágensek megvalósítása

Ebben a részben arra fogok kitérni, hogy hogyan lehet az ML-Agents használatával egymás ellen játszó ágenseket tanítani, mikre kell odafigyelni, és mik a szükséges lépések.

TeamID

Ha ellenséges viselkedésre szeretnénk ágenseket tanítani, akkor az első lépés, hogy az editoron keresztül beállítjuk a környezetben létező ágensek TeamID paraméterét a BehaviourParameters komponensen belül. Itt csak arra kell ügyelni, hogy a különböző csapatba tartozó ágensek különböző, az azonos csapatba tartozók pedig ugyanazon a TeamID-val rendelkezzenek. Amennyiben több ugyanolyan környezeten is párhuzamosan tanítjuk az ágenseket, nincs szükség minden környezetben egyedi TeamID-k használatára. Ebben a projektben a piros csapat a nullás, a sárga csapat pedig az egyes ID-t kapta. Itt fontos kiemelni, hogy ez az azonosító lényegében semmiben sem befolyásolja egy adott ágens tanítását, tehát például nem kapják meg implicit megfigyelésként, hogy melyik csapatba tartoznak. A TeamID a később tárgyalt self-play funkcionál játszik szerepet. A tréner ugyanis ez alapján tudja beazonosítani az egy csapatba tartozó ágenseket.

Szimmetrikus megfigyelések

Itt szeretnék kitérni egy fontos kérdésre, mégpedig, hogy a tanítás során hogyan tudjuk elérni, hogy a különböző csapatok különbözően viselkedjenek, például jelen projekt esetében a két különböző csapat különböző tagekkel ellátott objektumokat támadjon (a piros csapatnak a sárga ágenseket, a sárga csapatnak a pirosakat kell támadnia). A naiv megközelítés talán az lenne, hogy az ágensek legyenek tisztába azzal, hogy melyik csapatba tartoznak (pl. teamID hozzáadása a megfigyelésekhez) és ettől függően tanulják meg, hogy mi a feladatuk. Ennek a módszernek a hátránya, hogy igazából teljesen fölöslegesen várunk el egy plusz hozzárendelést az ágensektől, ráadásul ez megadja a lehetőséget az esetleges viselkedésbeli aszimmetria kialakulására csapatok között. Egy kevésbé egyértelmű, de talán annál hatékonyabb megoldás, ha a két csapat ágensei igazából teljesen ugyanazon policy alapján hozzák a döntéseket, és csak az ágensek megfigyeléseit módosítjuk a két csapat között a kívánt viselkedés elérésének megfelelően.

Vegyük például ebben a projektben azt a megfigyelést, ami a pályán lévő kontroll pont aktuális állapotát közli az ágensekkel. A naiv megközelítés szerint rendelnünk kéne egy-egy értéket a kontroll pont minden lehetséges állapotához. Pl.:

- -1 – a kontrollpontot a piros csapat birtokolja

```
CTState currentState = envController.ctState;
int stateObs = 0;
switch (currentState)
{
    case CTState.Red:
        stateObs = team == Team.Red ? 1 : -1;
        break;
    case CTState.Yellow:
        stateObs = team == Team.Yellow ? 1 : -1;
        break;
}
```

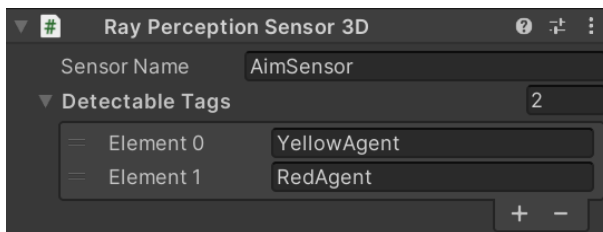
15. ábra: A kontroll pont állapotának átadása megfigyelésként

- 0 – a kontrollpontot egyik csapat sem birtokolja
- 1 – a kontrollpontot a sárga csapat birtokolja

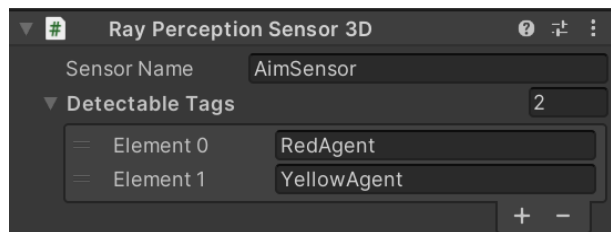
Ebben az esetben nyilvánvaló, hogy a két csapat ágenseinek nem ugyanaz az érték jelzi a kívánt állapotot. Ekkor csak úgy lehet egyáltalán sikeres a tanítás, ha az ágensek a megfigyeléseiket használva tisztában vannak azzal, hogy melyik csapat tagjai. Ez persze egy felesleges plusz feladatot hoz be az ágensek számára, ráadásul nincs garancia rá, hogy a két csapat ugyanolyan eredményeket fog tudni elérni/ugyanolyan ütemben tud tanulni. A hatékonyabb megoldás persze az, ha a két csapat számára minden megfigyelés ugyanazt a jelentést hordozza, tehát szimmetrikus. Ebben a példában én a következő módon rendelek értékeket a kontroll pont állapotaihoz:

- -1 – a kontroll pont az ellenséges csapat birtokában van
- 0 – a kontroll pontot egyik csapat sem birtokolja
- 1 – kontroll pontot az ágens saját csapata birtokolja

Ebben az esetben az ágenseknek nem szükséges tudniuk csapatuk ID-ját, és biztos, hogy a két csapat teljesítménye nem fog eltérni egymástól, hiszen ugyanazt a cél állapotot próbálják elérni.



16. ábra: A piros csapat szenzorai



17. ábra: A sárga csapat szenzorai

Itt fontos megjegyezni, hogy nem csak a vektor alapú megfigyeléseket szükséges szimmetrikussá tenni, hanem például a RayPerceptionSensor-ból érkező megfigyeléseket is! Ez a gyakorlatban azt jelenti, hogy a RayPerceptionSensor által felismerhető tageket különböző sorrendbe kell beírni a felismerhető tagek listájába (azon tageknél, amiket érzékelve különböző viselkedést várunk el a csapatoktól). Tehát például, ha az egyik csapat ágenseinek szenzorjai a 0. indexen érzékelik az ellenséges objektumokat, akkor ügyelni kell rá, hogy a másik csapatnál is ilyen módon állítsuk be a szenzorokat. Ez azért működik, mert ezek a szenzorok egy taggal ellátott objektum érzékelésekor nem az adott tag globális ID-ját továbbítják megfigyelésként, hanem az adott tag indexét az editorban beállított felismerhető tagek listájában. Ezt szemlélteti a 13. és 14. ábra.

Self-Play

Ha egy tanításon belül szeretnénk több különböző csapathoz tartozó ágenseket egymás elleni viselkedésre tanítani, akkor a hagyományos megközelítés, miszerint minden ágens a tanítási folyamat aktuálisan legfrissebb policyját használja, nem vezetne jó eredményre. Ennek az oka, hogy egy ágens teljesítménye teljes mértékben az ellenséges ágensek képességeitől függ. Ez azért probléma, mert minden csapat ágensei ugyanolyan sebességgel fejlődnek és változnak, minden tanítási iteráció után más feladattal kellene megbirkóznia az ágensnek. Ez lényegében teljesen ellehetetleníti a folyamatos és konzisztens tanulást.

Gondoljunk arra a szituációra, amikor a tanítás folyamán egyszerre tanítjuk mind a két csapatot, és mind a két csapat ágensei az aktuális, legfrissebb policy alapján hozzák a döntéseiket. Tegyük

fel, hogy az aktuális epizódot megnyerte a piros csapat. Itt merül fel a kérdés, hogy ebből az eredményből mit is lehet valójában leszűrni/mit is tanulhatnak az ágensek pontosan ebből? Mivel mind a két csapat ugyanazon policy alapján hozta a döntéseket, így egyáltalán nem egyértelmű, hogy milyen irányba kéne folytatni a tanulást ezen a ponton (hiszen az ágens lényegében legyőzte saját magát) és nem egyértelmű, hogy jó irányba léptünk-e az előző tanítási iterációban.

Erre a problémára kínál megoldást a Self-Play-nek nevezett módszer. A módszer alap ötlete egyszerű. Ahelyett, hogy egyszerre tanítanánk minden csapatot és minden ágens a legfrissebb policyt használja, tanítsunk egyszerre mindig csak 1 csapatot, a többi csapat ágensei pedig használják a policy/modell egy régebbi változatát. Bizonyos lépésközönként pedig cseréljessük az éppen tanított csapatot. Ez a megoldás érezhetően sokkal egyértelműbbé teszi a tanulást és a modell kiértékelését. Hiszen, ha az éppen tanított csapat, az aktuális policyt használva legyőzi a másik, régebbi policyt használó csapatot, akkor ebből egyértelműen látszódik a fejlődés és valószínűleg jó irányba haladt a tanítás.

Az ML-Agents olyan módon implementálja ezt a módszert, hogy bizonyos lépésközönként készít egy pillanatképet a modell aktuális állapotáról. Ezekből a pillanatképekből nyilvántartja az utolsó pár mentést, és közülük random módon sorsolja ki, hogy az éppen nem tanított csapat melyiket használja. A Self-play beállítását a config file-ban az 15. ábra szemlélteti.

```
self_play:  
  save_steps: 50000  
  team_change: 200000  
  swap_steps: 20000  
  window: 10  
  play_against_latest_model_ratio: 0.5  
  initial_elo: 1200.0
```

18. ábra: Self-play a config file-ban

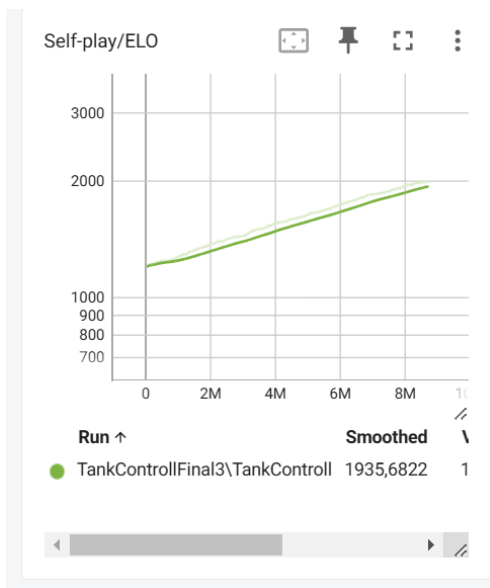
Fontos odafigyelni rá, hogy ezek a beállítások (pl.: save-steps, swap-steps) nagymértékben befolyásolják a tanítás stabilitását. Ezeknek a hiperparamétereknek a beállításánál ügyelni kell rá, hogy az éppen nem tanított csapat által használt policy-ket tartalmazó buffer minél nagyobb részét fedje le a tanításnak (minél messzebbre nyúljon vissza). Ezt úgy tudjuk elérni, hogy a save_steps és window paraméter értékét megnöveljük, ugyanakkor fontos kiemelni, hogy ezek a módosítások mindig növelni fogják a tanítás idejét. A másik fontos paraméter a swap_steps, ami azt állítja, hogy az éppen nem tanított csapat milyen sűrűn cserélje le az aktuálisan használt policy-t. Ezt az értéket úgy kell beállítani, hogy elég időt kapjon a tanuló csapat arra, hogy megtanulja az adott policy-t legyőzni, viszont még azelőtt váltson az ellenséges csapat mielőtt a modell túltanulna az aktuális ellenség stratégiájának legyőzésére.

ELO

Egymás ellen dolgozó ágensek tanításánál, ahogy már említettem, egy adott ágens teljesítménye az ellenséges csapat képességeitől és tevékenységétől függ. Emiatt a tanítás sikerességét nem lehet a megszokott metrikákkal vizsgálni. A hagyományosan használt CumulativeReward, illetve GroupCumulativeReward metrikák ugyanis ilyen esetben a 16. ábrán láthatóhoz hasonló módon fognak viselkedni.



19. ábra: Reward metrikák self-play használata mellett



20. ábra: ELO diagram képe sikeres tanítás esetén

A Unity ML-Agents pontosan emiatt a probléma miatt egy egyedi, kizárólag erre a felhasználási esetre működő metrikát is támogat, ez az ELO. A self-play opciót engedélyezve az ELO metrika veszi át a hagyományos rewardok helyét a tanulás során.

Az ELO egy mérőszám, ami a kiindulási állapothoz képesti relatív teljesítményt modellezi egy null-összegű játszmában. Nagyvonalakban az ELO növekedni fog, amennyiben az aktuálisan tanított csapat legyőzi az ellenfelet, különben csökken. A növekedés/csökkenés mértéke függ az epizód során tanított ágensek teljesítményétől (csapat és egyéni összesített jutalmaktól), valamint a két egymással szemben álló csapat ELO-béli differenciájától. Például, ha egy csapat legyőz egy hozzá ELO-ban közel álló csapatot, akkor viszonylag nagy mértékű lesz a növekedés, viszont amennyiben például

döntetlent játszik egy elvileg sokkal gyengébb csapat ellen, akkor nagy mértékben esni fog a mérőszám. Az epizód elején tehát a 18. ábrán látható módon a keretrendszer kiszámolja a csapat várható pontszámát, majd az epizód végeztével frissíti az aktuális ELO számot attól függően, hogy a várható pontszámhoz képest hogyan teljesített a csapat (19. ábra).

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

Expected score of player A

Difference between Elo score of B and A

21. ábra: A csapat/ágens várható pontszámának számítása

$$R'_A = R_A + K(S_A - E_A)$$

New Elo score of player A	Current Elo score of A	Adjustement Rating (K-factor)	Performed score of A	Expected score of A
---------------------------------	------------------------------	-------------------------------------	-------------------------	------------------------

22. ábra: Csapat/ágens új ELO értékének számítása

Ahhoz, hogy az ELO számítás jól működjön, be kell tartani pár szabályt a jutalmak kiosztásakor. Fentebb már említettem, hogy az ELO igazából null-összegű játékokra működik jól (a két csapat végső pontszámainak összege nulla), a tapasztalataim azonban azt mutatják, hogy elegendő, ha a kör végén a győztes csapat pozitív, a vesztes pedig negatív összesített jutalommal zárja az epizódot. Amire viszont ténylegesen oda kell figyelni, hogy az ELO számítás a csapatoknak utoljára kiosztott jutalom alapján fogja meghatározni a győztest. Fontos tehát, hogy az epizód legvégén a legutolsó jutalom, amit adunk a csapatoknak az az alábbi módon nézzen ki:

- 1 – a győztes csapatnak
- -1 – a vesztes csapatnak
- 0 – döntetlen esetén mindkét csapatnak

A Unity ML-Agents dokumentációjában külön kérnek minden fejlesztőt, hogy self-play használatkor lehetőleg minél egyszerűbb jutalmazási rendszert alkalmazzunk, nehogy véletlenül elrontsuk az ELO számítást.

A tanítási folyamat

Előtanítás

A projektben az ágensek által megoldandó feladat jellege miatt a kiosztható jutalmak igen ritkásak. Az ágensek célja, hogy az ellenfél tankjait elpusztítsák és elfoglalják a kontroll pontot. Ebből kiindulva intuitívan pozitív jutalmat ellenséges tank meglövésekor, illetve a kontroll pont elfoglalásakor lehetne adni. Érezhetően a hagyományos trial & error elvet követő tanulási módszer ebben az esetben nem lenne túl hatékony, hiszen az eleinte teljesen random akciók mellett nagyon kis valószínűséggel következne be bármelyik jutalmazható esemény. Az tovább nehezíti a feladatot, hogy a tank jármű irányítása már önmagában is egy viszonylag nehéz, sok dimenziós feladat. Persze kisebb állapotfüggő „mikro jutalmakkal” talán rá lehetne vezetni az ágenseket a kívánt működésre (pl.: büntetés minden lépésben a kontroll ponttól való távolság függvényében). Azonban a témában szerzett eddigi tapasztalataim alapján nem jó gyakorlat jutalmakkal túlszabályozni a környezetet. Ezen felül ahogy fentebb az ELO számításról szóló részben említettem, self-play használatkor fokozottan kell ügyelni a kiszabott jutalmakra/büntetésekre. A tanítás elején a legnehezebb feladat arra rávenni az ágenseket, hogy felderítsék a játéktér és ezáltal megtalálják a pálya közepén található kontrollpontot.

Szerencsére az ML-Agents a hagyományos „Extrinsic reward” -on kívül számos olyan módszert támogat, amik kifejezetten az ilyen környezetekben nyújtanak nagy segítséget. Ilyen például a curiosity, ami az állapotter felderítésére ösztönzi az ágenseket, vagy az imitation learning különböző szintén beépített fajtái. A probléma már csak az, hogy a kooperatív ágensek tanítására szolgáló POCA tréner ezen módszerek egyikét sem támogatja.



23. ábra: Az előtanításra használt környezet

A problémát úgy oldottam meg, hogy az ágenseket első körben egy 1v1-es tanító környezetben tanítottam a PPO trénerrel használva, ami már támogatja a curiosity és az imitation learning alkalmazását. A projekt korábbi verziójában az előtanításnál GAIL³-t használtam egy általam lejátszott játék kört használva demonstrációként. Ezzel az volt a probléma, hogy a GAIL még a „strength” paraméter csökkentése után is egy nagyon erős megerősítést nyújt az ágens felé, ami miatt könnyen túl tanulhatnak a demonstráció esetleges tökéletlenségeire. A mostani verzióban szerettem volna elérni, hogy az ágensek maguktól jöjjenek rá a játék és a tank irányításának alapjaira is. Ehhez az alap Extrinsic rewardokon kívül még az RND⁴ reward signalt használtam. Az RND nagyon hasonlóan működik a Curiosity-hoz, tehát ez is a felfedezésre buzdítja az ágenseket. A különbség az, hogy amíg a Curiosity az új állapot átmenetek felderítését szorgalmazza, addig az RND pusztán az eddig nem ismert állapotok (megfigyelések) elérését jutalmazza. Ezen kívül az RND implementációja valamivel egyszerűbb is. Ez a reward signal jól alkalmazkodik azokhoz a feladatokhoz, amikor a célunk

³ Generative adversarial imitation learning

⁴ Random Network Distillation

pusztán az, hogy az ágens felderítse a környezetét, új helyekre és új helyzetekbe keveredjen. Ezekben az esetekben ez a módszer tipikusan gyorsabban, hatékonyabban oldja meg a feladatot.

```
behaviors:
  TankControll:
    trainer_type: ppo
    hyperparameters:
      batch_size: 4096
      buffer_size: 81920
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 3
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
      rnd:
        gamma: 0.99
        strength: 0.01
    keep_checkpoints: 5
    checkpoint_interval: 1000000
    max_steps: 30000000
    time_horizon: 900
    summary_freq: 30000
    self_play:
      save_steps: 100000
      team_change: 500000
      swap_steps: 10000
      window: 10
      play_against_latest_model_ratio: 0.5
      initial_elo: 1200.0
```

Egyéni jutalmak
+0.5 a kontroll pont elfoglalásakor
+0.1 ellenséges taggel ellátott objektum eltalálásakor
-0.01 nem ellenséges taggel ellátott objektum eltalálásakor
-0.05 ütközés esetén

24. ábra: Az előtanítás konfigurációja

Csapatszintű tanítás

Az előtanítást követően az ágensek már nagyjából tisztában voltak a megoldandó feladat alapjaival, megtanulták, hogy milyen tagekre kell célozni és löni, és hogy a kontroll pont közelébe kell jutniuk ahhoz, hogy elfoglalják azt. Emiatt a továbbiakban már nem volt szükség egyéb, kiegészítő reward signalra, így át tudtam térni a tényleges feladat betanítására, az 5v5-ös környezetre és a MA-POCA tréner használatára. Ez egy nagyon hosszú tanítási folyamat volt, ami során az ágensek elsajátították a játék megnyeréséhez szükséges lehető leghatékonyabb stratégiát és a csapatmunka alapjait. Ebben a tanításban a jutalmak számát is minimálisra csökkentettem, a Unity javaslatait követve a csapatok már csak az epizód végén kaptak 1-1 jutalmat attól függően, hogy nyertek vagy veszítettek, és megkorra pontkülönbséggel tették azt. Ez a nagyon ritkás jutalmi rendszer persze jócskán meghosszabbította a tanítás folyamatát, cserébe viszont megadta a lehetőséget a tanító algoritmusnak, hogy az ágensek teljesen szabadon találják meg a lehető leghatékonyabb stratégiát.



25. ábra: 5v5-ös tanító környezet

Mivel a self-play használata még instabilabbá teheti az amúgy sem túl stabil megerősítéses tanulás folyamatát, így a hiperparaméterek beállításánál a stabilitás elérése volt a fő cél, még akkor is, ha ez a tanulási idő rovására megy. Ez többek között a magas buffer_size, batch_size, save_steps paramétereknél jelenik meg, ahogy az a 23. ábrán látható. A hiperparaméterek beállításánál ügyelni kellett arra is, hogy egy teljes epizód lejátszása egy darab 5v5-ös környezetben összességében sokkal több tanító lépésből állt (trainer steps) hiszen egyszerre nem 1, hanem 5 darab ágens tanult minden egyes környezetben. Ez azért fontos, mert az előtanítás során például egy 40 000 körüli buffer size azt jelentette, hogy a bufferben benne van kb. 40-50 teljes lejátszott epizód, az 5v5-ös környezetben azonban egy ekkora buffer már csak kb. 10 epizódot tartalmazna. Ezt minden olyan paraméter beállításánál figyelembe kellett venni, ami a trainer step-ek számán alapul. Mivel a self-play miatt a tanítás során a megoldandó feladat lényegében folyamatosan nehezedik, hiszen az ellenfél csapat is folyamatosan javul, így ez

lényegében egy fajta Curriculum learning eset. Ilyen esetekben szokás a learning-rate, beta, epsilon paramétereket konstans értéken hagyni a tanítás során.

Csapatszintű jutalmak
+0.5 + csapatok pontszámai közötti különbség (0 és 0.5 közé skálázva) a nyertes csapatnak
+0.0 döntetlen esetén
-0.5 + csapatok pontszámai közötti különbség (0 és 0.5 közé skálázva) a vesztes csapatnak

```

1  behaviors:
2      TankControll:
3          trainer_type: poca
4          hyperparameters:
5              batch_size: 4096
6              buffer_size: 81920
7              learning_rate: 0.0001
8          beta: 0.003
9          epsilon: 0.2
10         lambd: 0.95
11         num_epoch: 3
12         learning_rate_schedule: constant
13     network_settings:
14         normalize: false
15         hidden_units: 512
16         num_layers: 3
17         vis_encode_type: simple
18     reward_signals:
19         extrinsic:
20             gamma: 0.995
21             strength: 1.0
22     keep_checkpoints: 5
23     checkpoint_interval: 1000000
24     max_steps: 100000000
25     time_horizon: 900
26     summary_freq: 30000
27     self_play:
28         save_steps: 150000
29         team_change: 750000
30         swap_steps: 15000
31         window: 25
32         play_against_latest_model_ratio: 0.25
33         initial_elo: 1200.0
34

```

26. ábra: Végső tanítás konfigurációja

Fejlesztési lehetőségek

A fentebb részletezett tanítási folyamatot követően az eseten többségében az ágensek teljesítménye megközelíti egy humán játékosét. Ennek ellenére még akadnak olyan szituációk, amikor a modell nem találja a logikus megoldást. A csapatmunka terén is egyértelműen van még hova fejlődni, bár a játék mostani állapotában, ahol csak tankok szerepelnek a színtérben, az, hogy mit értünk csapatmunka alatt annyira nem is egyértelmű. Ezeken a javításokon kívül az alábbi ötleteket megvalósítva szeretném a későbbiekben fejleszteni a játékot:

- Több különböző jármű implementálása és tanítása (különböző feladatok elvégzésére). Például repülő járművek, légvédelmi járművek, drónok, amik „gyógyítják” sérült csapattársaikat stb...
- Az ágensek képesek legyenek például a humán játékostól érkező parancsok végrehajtására
- Ágensek képesek legyenek valamilyen módon kommunikálni egymással (esetleg nyelvi modell segítségével)